



**UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

Artur Schmidt

Joshua Pires

Elisson Amaral

Antonio Marcos

DOCUMENTAÇÃO TÉCNICA
Sistema UTI Care — Gestão Especializada de UTIs

FORTALEZA

2025

Artur Schmidt

Joshua Pires

Elisson Amaral

Antonio Marcos

DOCUMENTAÇÃO TÉCNICA

Sistema UTI Care — Gestão Especializada de UTIs

Este documento contém a documentação técnica do Sistema UTI Care desenvolvido na componente curricular N392 - Projeto Aplicado Plataformas Web como requisito para obtenção de nota.

Supervisor: Prof. Bruno Lopes, Me

FORTALEZA

2025

SUMÁRIO

1. INTRODUÇÃO	4
1.1. CONTEXTO E JUSTIFICATIVA.....	4
1.2. OBJETIVOS	4
1.3. ESCOPO E DELIMITAÇÃO	5
2. ENGENHARIA DE REQUISITOS	6
2.1. REQUISITOS FUNCIONAIS (RFs)	6
2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)	7
3. PROJETO E ARQUITETURA DO SOFTWARE	9
3.1. ARQUITETURA GERAL	9
3.2. PROJETO DO BANCO DE DADOS	10
3.3. PROJETO DE API (SERVER ACTIONS)	12
4. TECNOLOGIAS E FERRAMENTAS	14
4.1. STACK DE TECNOLOGIAS	14
4.2. FERRAMENTAS DE DESENVOLVIMENTO	15
5. IMPLEMENTAÇÃO E RESULTADOS	16
5.1. TELAS DO SISTEMA	16
6. AMBIENTE E GUIA DE IMPLANTAÇÃO	18
6.1. REQUISITOS DO AMBIENTE	18
6.2. PROCESSO DE IMPLANTAÇÃO	18
6.3. ACESSO À APLICAÇÃO IMPLANTADA	19
7. CONCLUSÃO	20
7.1. TRABALHOS FUTUROS	20
7.2. LIÇÕES APRENDIDAS	20

1. INTRODUÇÃO

1.1.CONTEXTO E JUSTIFICATIVA

As Unidades de Terapia Intensiva (UTIs) são ambientes de alta complexidade clínica, nos quais a precisão e a agilidade no registro e na consulta de informações sobre o estado dos pacientes são determinantes para a qualidade do cuidado. Em muitos hospitais brasileiros, especialmente em instituições de médio porte, o gerenciamento de leitos e o registro das evoluções clínicas ainda dependem de processos manuais: planilhas físicas, prontuários em papel e comunicação verbal entre plantões. Essa fragmentação resulta em riscos concretos: perda de informação clínica crítica na transição de turno, demora na identificação de leitos vagos e dificuldade de auditoria das condutas médicas adotadas.

O sistema **UTI Care** surge como resposta direta a esse problema. Trata-se de uma plataforma web desenvolvida para centralizar a gestão operacional e clínica de UTIs, oferecendo ao médico intensivista um ambiente digital estruturado para registrar evoluções clínicas multi-sistemas, ao enfermeiro um painel de controle do status dos leitos em tempo real e ao gestor hospitalar uma visão estratégica da ocupação e das atividades da unidade.

O público-alvo do sistema são hospitais com UTIs ativas que buscam digitalizar seus processos assistenciais sem necessidade de adquirir sistemas proprietários de alto custo (como os módulos de UTI de sistemas como o Tasy ou o MV). O UTI Care se posiciona como uma solução acessível, focada e de fácil implantação.

1.2. OBJETIVOS

Objetivo Geral

Desenvolver uma aplicação web full-stack para gerenciar, de forma centralizada e digital, os leitos, pacientes e evoluções clínicas de uma Unidade de Terapia Intensiva, oferecendo interfaces diferenciadas para médicos, enfermeiros e gestores hospitalares.

Objetivos Específicos

- Implementar um painel de controle de leitos com visualização em tempo real dos status (Vago, Ocupado, Higienização, Manutenção) e ações de admissão e alta de pacientes.
- Desenvolver um formulário de evolução clínica multi-sistemas (respiratório, neurológico, hemodinâmico, renal, nutricional, hematoinfeccioso e profilaxias) com geração automática de texto narrativo estruturado.
- Criar um prontuário digital do paciente com histórico de evoluções e dados de identificação editáveis.
- Implementar um sistema de autenticação com controle de acesso baseado em papéis (RBAC), diferenciando os fluxos de Médico, Enfermeiro, Administrador e Gestor.

- Disponibilizar um painel gerencial com indicadores de ocupação e feed de atividades da UTI.
- Garantir a segurança dos dados com sessões criptografadas via JWT e senhas com hash bcrypt.

1.3.ESCOPO E DELIMITAÇÃO

Escopo (o que o sistema FAZ):

- Cadastro, edição, visualização e exclusão de leitos de UTI.
- Admissão e alta de pacientes com registro de data e observações clínicas iniciais.
- Registro de evoluções clínicas estruturadas em 7 sistemas fisiológicos.
- Geração automática de texto narrativo a partir dos dados da evolução.
- Controle de acesso baseado em papéis (DOCTOR, NURSE, ADMIN, MANAGER) via middleware.
- Autenticação com sessão JWT assinada e armazenada em cookie seguro.
- Painel gerencial com KPIs de ocupação e lista de pacientes internados.
- Histórico de evoluções por paciente/leito.
- Cadastro de novos usuários do sistema.

Delimitação (o que o sistema NÃO FAZ):

- O sistema não emite prescrições médicas eletrônicas com validade legal (e-Prescrição).
- Não há integração com sistemas de faturamento hospitalar (TISS/TUSS/SADT).
- Não há integração com equipamentos de monitorização à beira do leito.
- O sistema não gera laudos ou relatórios médicos com validade de prontuário legal.
- Não há módulo financeiro (folha de pagamento, contas a pagar/receber).
- Não há aplicativo mobile nativo; o acesso é via navegador web.
- O sistema não cobre a gestão de farmácia, estoque de medicamentos ou insumos.

2.ENGENHARIA DE REQUISITOS

2.1.REQUISITOS FUNCIONAIS (RFs)

ID	Nome do Requisito	Descrição
RF01	Autenticar Usuário	O sistema deve permitir que usuários cadastrados acessem a plataforma mediante e-mail e senha válidos. Após autenticação, o sistema deve redirecionar o usuário para o painel correspondente ao seu papel (role).
RF02	Registrar Novo Usuário	O sistema deve permitir o cadastro de novos usuários informando nome, e-mail, senha e papel (DOCTOR, NURSE, ADMIN, MANAGER). A senha deve ser armazenada com hash bcrypt.
RF03	Gerenciar Leitos	O sistema deve permitir criar novos leitos de UTI com número e rótulo, e visualizar todos os leitos com seus status atuais (Vago, Ocupado, Higienização, Manutenção).
RF04	Admitir Paciente	O sistema deve permitir admitir um paciente em um leito vago, registrando nome e observações iniciais. O leito deve ter seu status atualizado automaticamente para "Ocupado".
RF05	Realizar Alta de Paciente	O sistema deve permitir registrar a alta de um paciente, desvinculando-o do leito e alterando o status do leito para "Em Higienização" automaticamente.
RF06	Gerenciar Dados do Paciente	O sistema deve permitir visualizar e editar os dados cadastrais do paciente internado (nome, data de nascimento, altura, peso, gênero, observações).
RF07	Registrar Evolução Clínica	O sistema deve permitir ao médico registrar uma evolução clínica completa com 7 sub-formulários (Respiratório, Neurológico, Hemodinâmico, Renal, Nutricional, Hematoinfeccioso, Profilaxias), associada ao leito e ao paciente.
RF08	Gerar Texto de Evolução	O sistema deve gerar automaticamente um texto narrativo estruturado a partir dos dados preenchidos no formulário de evolução, consolidando todos os sistemas fisiológicos em um único documento textual.
RF09	Visualizar Histórico de Evoluções	O sistema deve exibir o histórico de evoluções clínicas de um paciente, ordenadas da mais recente para a mais antiga, com data, horário e dados registrados.
RF10	Controle de Acesso por Papel (RBAC)	O sistema deve garantir que cada papel de usuário acesse somente as rotas autorizadas: DOCTOR e NURSE acessam /dashboard; ADMIN e MANAGER acessam /admin. Tentativas de acesso a rotas não autorizadas devem resultar em redirecionamento.
RF11	Painel Gerencial com KPIs	O sistema deve exibir ao gestor indicadores de ocupação da UTI (total de leitos, ocupados, vagos, em higienização) e uma lista dos pacientes internados no dia.

ID	Nome do Requisito	Descrição
RF12	Gerenciar Status de Limpeza do Leito	O sistema deve permitir que o profissional registre a conclusão da higienização de um leito, retornando seu status para "Vago" e disponibilizando-o para nova admissão.

2.2.REQUISITOS NÃO FUNCIONAIS (RNFs)

Segurança:

- **RNF01:** As senhas dos usuários devem ser armazenadas no banco de dados exclusivamente como hash produzido pelo algoritmo bcrypt com fator de custo mínimo de 10 rounds. Senhas em texto plano nunca devem ser persistidas.
- **RNF02:** As sessões de usuário devem ser gerenciadas por tokens JWT assinados com chave secreta (HS256), armazenados em cookies `HttpOnly` para prevenir acesso via JavaScript (mitigação de XSS).
- **RNF03:** O middleware de RBAC deve adotar uma política de negação padrão (*fail-closed*): qualquer papel não mapeado explicitamente nas regras de acesso deve ter acesso negado.
- **RNF04:** A comunicação entre o cliente e o servidor em produção deve ocorrer exclusivamente via HTTPS.

Desempenho:

- **RNF05:** O painel de leitos (`/dashboard`) deve carregar e exibir o grid completo de leitos em menos de 2 segundos em uma conexão de 10 Mbps, considerando até 30 leitos.
- **RNF06:** As Server Actions de leitura devem retornar resposta em menos de 800ms para consultas sem agregações complexas.

Usabilidade:

- **RNF07:** O formulário de evolução clínica deve ser organizado em etapas (stepper) com no máximo 7 passos, evitando que o médico precise rolar uma única página longa.
- **RNF08:** A interface deve ser responsiva e utilizável em telas com resolução mínima de 1280×720px.

Compatibilidade:

- **RNF09:** A aplicação deve ser renderizada e funcionar corretamente nas duas últimas versões estáveis dos navegadores Google Chrome, Mozilla Firefox e Microsoft Edge.
- **RNF10:** O ambiente de execução do servidor deve ser Node.js versão 22.22.2 ou superior.

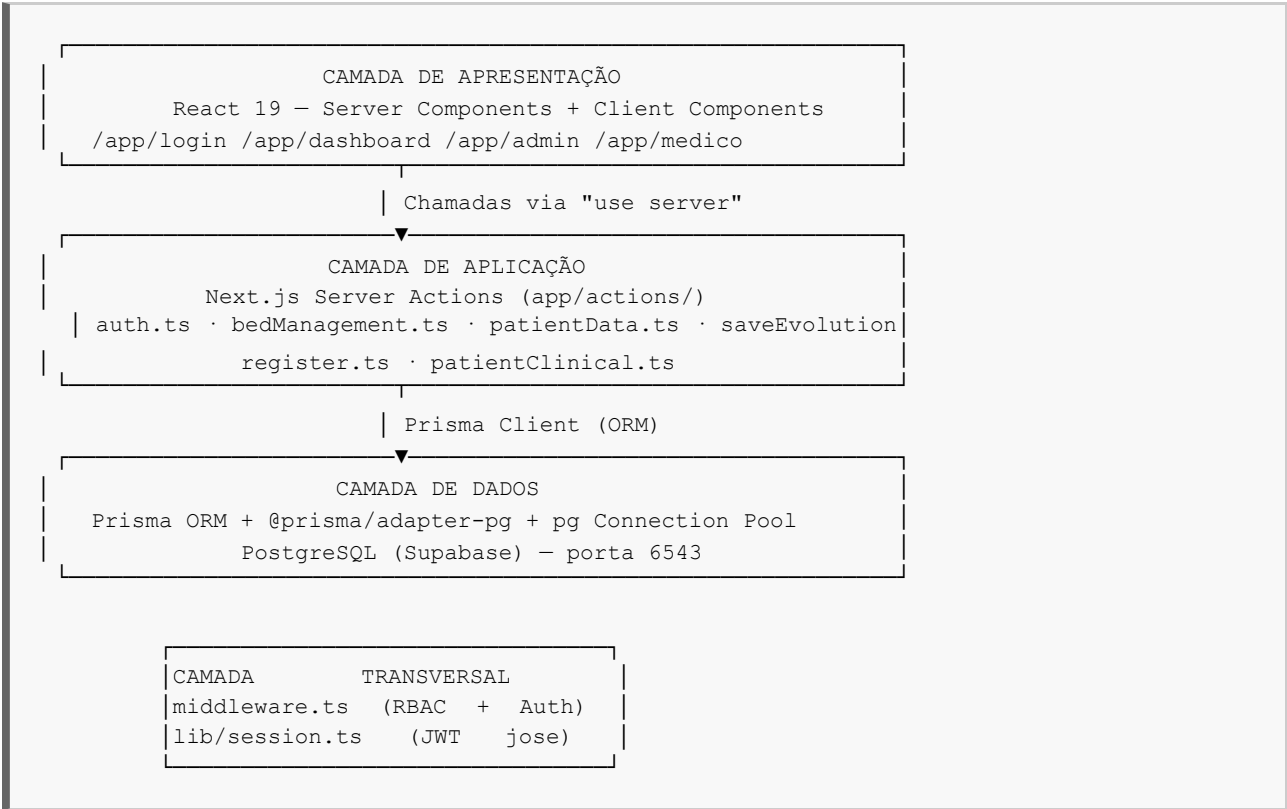
Manutenibilidade:

- **RNF11:** O código deve ser escrito em TypeScript com tipagem estrita, garantindo verificação estática de tipos em toda a base de código.
- **RNF12:** Os componentes de formulário críticos devem possuir testes unitários automatizados cobrindo os casos de renderização e submissão.

3.PROJETO E ARQUITETURA DO SOFTWARE

3.1.ARQUITETURA GERAL

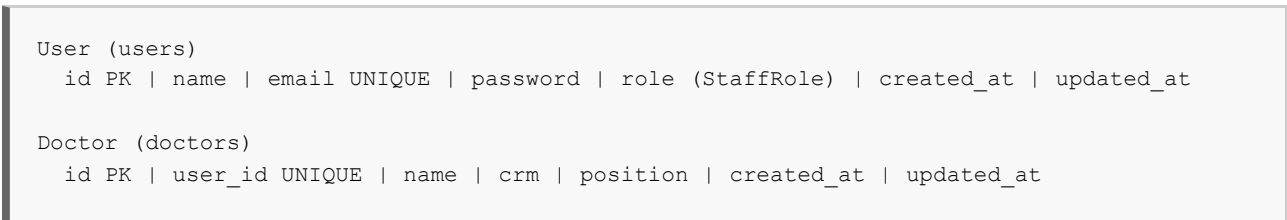
O UTI Care foi projetado utilizando uma **arquitetura full-stack monolítica baseada em Next.js 16 (App Router)**, organizada em camadas lógicas bem definidas:



Justificativa do padrão arquitetural: A escolha do Next.js como framework full-stack elimina a necessidade de manter dois repositórios separados (frontend React + backend Node.js/Express), reduzindo a complexidade operacional do projeto. As **Server Actions** funcionam como a camada de aplicação (equivalente a um conjunto de endpoints RPC), sendo executadas inteiramente no servidor. O Prisma ORM, combinado com o adapter `@prisma/adapters-pg`, permite usar um pool de conexões (`pg.Pool`) compatível com ambientes serverless (Vercel/Supabase), evitando esgotamento de conexões. O **middleware de RBAC** intercepta todas as requisições antes de chegarem às páginas, implementando uma política de segurança fail-closed.

3.2.PROJETO DO BANCO DE DADOS

Diagrama Entidade-Relacionamento (Textual):



```
Patient (patients)
  id PK | name | birth_date? | height? | weight? | gender? (Sex)
  | commentary? | arrival_date? | admission_date | discharge_date?
  | created_at | updated_at

Bed (beds)
  id PK | bed_number UNIQUE | label? | type?
  | status (BedStatus) | current_patient_id? FK→Patient.id UNIQUE
  | created_at | updated_at

ClinicalEvolution (clinical_evolution)
  id PK | patient_id FK | bed_id FK | doctor_id FK
  | [campos respiratório, neurológico, hemodinâmico, renal,
    nutricional, hematoinfeccioso, profilaxias]
  | generated_text? | created_at | updated_at

DraftEvolution (draft_evolution)
  id PK | bed_id FK | doctor_id FK | patient_id? FK
  | form_data (JSON) | created_at | updated_at
  UNIQUE(bed_id, doctor_id)
```

Relacionamentos principais:

- `Bed ↔ Patient` : 1:1 via `current_patient_id` (unique) — um leito só pode ter um paciente ativo
- `Patient → ClinicalEvolution` : 1:N — um paciente pode ter muitas evoluções
- `Doctor → ClinicalEvolution` : 1:N — um médico pode registrar muitas evoluções
- `Bed → ClinicalEvolution` : 1:N — um leito possui histórico de evoluções

Enums do banco:

Enum	Valores
<code>BedStatus</code>	VACANT, OCCUPIED, CLEANING, MAINTENANCE
<code>Sex</code>	MALE, FEMALE, OTHER
<code>StaffRole</code>	DOCTOR, NURSE, ADMIN, MANAGER

Dicionário de Dados — Tabela `beds` :

Campo	Tipo	Chave	Nulo?	Descrição
<code>id</code>	INT (autoincrement)	PK	Não	Identificador único do leito
<code>bed_number</code>	INT	UNIQUE	Não	Número identificador do leito
<code>label</code>	VARCHAR	—	Sim	Rótulo legível (ex: "Leito 01")
<code>type</code>	VARCHAR	—	Sim	Tipo do leito (ex: "UTI Geral")

Campo	Tipo	Chave	Nulo?	Descrição
status	ENUM BedStatus	—	Não	Status atual do leito
current_patient_id	INT	FK / UNIQUE	Sim	Paciente atualmente internado; NULL = vago
created_at	TIMESTAMPTZ	—	Não	Data/hora de criação do registro
updated_at	TIMESTAMPTZ	—	Não	Data/hora da última atualização

Dicionário de Dados — Tabela `clinical_evolution` (campos principais):

Campo	Tipo	Descrição
id	INT PK	Identificador único da evolução
patient_id	INT FK	Vínculo com o paciente
bed_id	INT FK	Leito onde a evolução foi registrada
doctor_id	INT FK	Médico responsável pelo registro
airway_type	VARCHAR	Via aérea (fisiológica, tot, tqt)
respiratory_support	JSON	Suportes respiratórios ativos (array)
respiratory_spo2	VARCHAR	Saturação periférica de oxigênio (%)
neurologic_sedation	JSON	Drogas de sedação em uso (array)
neurologic_scales	VARCHAR	Escalas neurológicas (RASS, Glasgow)
hemodynamic_drugs	JSON	Drogas vasoativas em uso (array com doses)
hemodynamic_pam	VARCHA	Pressão Arterial Média (mmHg)
renal_diuresis	R	Diurese das últimas 24h (mL/kg/h)
renal_dialysis	VARCHAR	Modalidade de diálise, se aplicável
nutrition_support	VARCHAR	Suporte nutricional (Oral, Enteral, Parenteral)
hemato_antibiotics	JSON	Antibióticos em uso (nome, dose, via, dia)
hemato_cultures	JSON	Culturas coletadas e resultados
prophylaxis_tev	VARCHAR	Profilaxia de tromboembolismo venoso
prophylaxis_ibp	VARCHAR	Proteção gástrica (IBP)
generated_text	TEXT	Texto narrativo consolidado gerado automaticamente
created_at	TIMESTAMPTZ	Data/hora do registro da evolução

3.3.PROJETO DE API (SERVER ACTIONS)

O UTI Care não expõe uma API REST tradicional. Em vez disso, utiliza **Next.js Server Actions** — funções TypeScript marcadas com `"use server"` que são executadas exclusivamente no servidor e chamadas diretamente pelos componentes React. Esse padrão elimina a necessidade de rotas HTTP explícitas, reduzindo superfície de ataque e simplificando o desenvolvimento.

Arquivo:

`app/actions/auth.ts`

Action	Assinatura	Pré-condição	Pós-condição	Erros
<code>login</code>	(prevState, formData: FormData)	e-mail e senha não vazios; usuário existe no banco	Cria cookie de sessão JWT; redireciona para rota do papel	"Email e senha são obrigatórios" · "Credenciais inválidas"
<code>logout</code>	()	—	Apaga cookie de sessão; redireciona para /login	—

Arquivo: `app/actions/register.ts`

Action	Assinatura	Pré-condição	Pós-condição	Erros
<code>register</code>	(prevState, formData: FormData)	e-mail único; senha válida; papel válido	Cria User no banco com senha bcrypt; redireciona	E-mail duplicado · campos obrigatórios ausentes

Arquivo: `app/actions/bedManagement.ts`

Action	Assinatura	Pré-condição	Pós-condição	Erros
<code>getDashboardData</code>	()	—	Retorna array de leitos com paciente atual e última evolução	Erro de conexão com banco
<code>createBed</code>	(bedNumber: number)	bedNumber único	Cria leito com status VACANT; invalida cache	"Número de leito já existe"
<code>admitPatient</code>	(bedId, patientName, commentary?)	Leito deve existir	Cria Patient, vincula ao leito, status → OCCUPIED (transação atômica)	"Falha ao admitir paciente"
<code>dischargePatient</code>	(bedId: number)	Leito deve estar OCCUPIED	Desvincula paciente, registra discharge_date, status → CLEANING	"Falha ao dar alta"
<code>finishCleaning</code>	(bedId: number)	Leito deve estar CLEANING	Status → VACANT	"Falha ao finalizar"

Action	Assinatura	Pré-condição	Pós-condição	Erros
				"Falha ao excluir leito"
<code>deleteBed</code>	(bedId: number)	Leito deve estar VACANT	Remove leito do banco	

Arquivo: `app/actions/patientData.ts`

Action	Assinatura	Pré-condição	Pós-condição	Erros
<code>getBedDetails</code>	(bedId: number)	Leito deve existir	Retorna leito com dados completos do paciente atual	null se não encontrado
<code>getPatientFromBed</code>	(bedId: number)	Leito deve ter paciente	Retorna dados do paciente vinculado	null se vago
<code>getLastEvolution</code>	(patientId: number)	Paciente deve existir	Retorna última ClinicalEvolution ordenada por created_at desc	null se sem evolução
<code>savePatientData</code>	(patientId, data: Partial<Patient>)	Paciente deve existir	Atualiza campos do paciente; invalida cache	Erro de validação/banco

Arquivo: `app/actions/saveEvolution.ts`

Action	Assinatura	Pré-condição	Pós-condição	Erros
<code>saveEvolution</code>	(bedId: number, formData: EvolutionFormData)	Leito com paciente ativo; médico associado	Cria ClinicalEvolution com todos os campos mapeados; atualiza peso do paciente se informado	Erro de banco · leito sem paciente

Invariante: Todas as Server Actions retornam um objeto `{ success: boolean, error?:`

`string }` ou lançam redirecionamento via `redirect()`. Nenhuma action expõe stack traces ao cliente.

4.TECNOLOGIAS E FERRAMENTAS

4.1.STACK DE TECNOLOGIAS

Framework e Runtime:

- **Next.js 16 (App Router):** Escolhido como framework full-stack por unificar frontend e backend em um único projeto. O App Router permite uso de React Server Components e Server Actions, eliminando a necessidade de uma API REST separada.
- **React 19:** Utilizado para construção da interface do usuário com arquitetura de componentes reutilizáveis. A versão 19 introduz melhorias de performance e suporte nativo a Server Actions diretamente em formulários.
- **TypeScript 5:** Adotado em toda a base de código para verificação estática de tipos, reduzindo bugs em tempo de desenvolvimento e melhorando a documentação implícita do código.
- **Node.js ≥ 22.22.2:** Ambiente de execução do servidor, escolhido por sua maturidade, suporte a ES Modules e compatibilidade com o ecossistema Next.js.

Banco de Dados e ORM:

- **PostgreSQL (via Supabase):** Banco de dados relacional escolhido pela robustez transacional — essencial para operações atômicas como admissão de paciente. O Supabase oferece hospedagem gerenciada com suporte a pooler de conexões (PgBouncer), necessário para ambientes serverless.
- **Prisma ORM 7:** Escolhido pela geração automática de tipos TypeScript a partir do schema, o que garante que qualquer mudança no banco seja imediatamente refletida em erros de compilação, prevenindo inconsistências.
- **@prisma/adapters-pg + pg:** Adaptador de conexão que permite o uso de um pool de conexões PostgreSQL (pg.Pool), contornando a limitação de instâncias de conexão em deployments serverless na Vercel.

Autenticação e Segurança:

- **Jose 6 (JWT):** Biblioteca para criação e verificação de tokens JWT assinados (HS256). Utilizada no `lib/session.ts` para criar sessões criptografadas armazenadas em cookies `HttpOnly`, prevenindo acesso via JavaScript (mitigação XSS).
- **bcryptjs 3:** Implementação de bcrypt para hash de senhas no cadastro e comparação segura no login. Não expõe bindings nativos, sendo compatível com qualquer ambiente Node.js.

Interface e Estilos:

- **Tailwind CSS 4:** Framework de CSS utility-first utilizado para construir toda a interface de forma rápida e consistente. A versão 4 elimina a necessidade de arquivo de configuração

explícito.

- **Lucide React:** Biblioteca de ícones SVG com tipagem TypeScript, utilizada em todos os ícones da interface (leitos, status, ações clínicas).

Testes:

- **Jest** + **@testing-library/react:** Framework de testes unitários e de componentes.
Utilizado para testar os formulários de evolução clínica (renderização, preenchimento e submissão dos 7 sub-formulários).

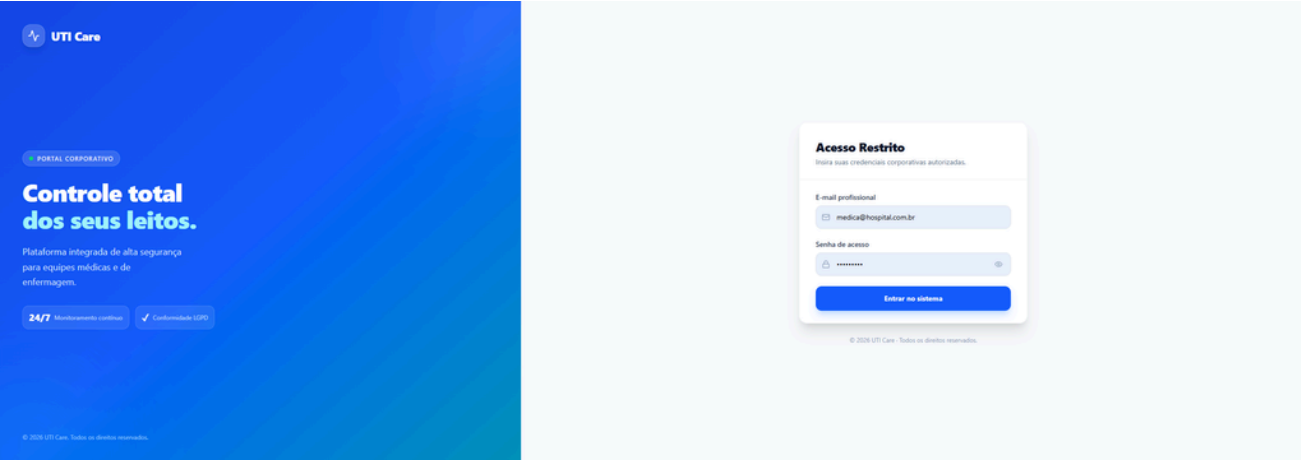
4.2.FERRAMENTAS DE DESENVOLVIMENTO

Ferramenta	Finalidade
Visual Studio Code	IDE padrão da equipe, com extensões para TypeScript, Tailwind CSS e Prisma
Git + GitHub	Controle de versão distribuído com repositório remoto no GitHub
Prisma Studio	Interface gráfica para inspeção e edição manual dos dados do banco PostgreSQL durante o desenvolvimento
Supabase Dashboard	Console web para gerenciamento do banco de dados, visualização de tabelas e configuração de variáveis de ambiente Execução da suite de testes unitários com suporte a coverage reports em HTML
Jest (CLI)	Linting estático do código TypeScript/React com regras específicas do ecossistema Next.js
ESLint + <code>eslint-config-next</code>	
npm	Gerenciador de pacotes e executor de scripts (dev, build, test, test:coverage)

5.IMPLEMENTAÇÃO E RESULTADOS

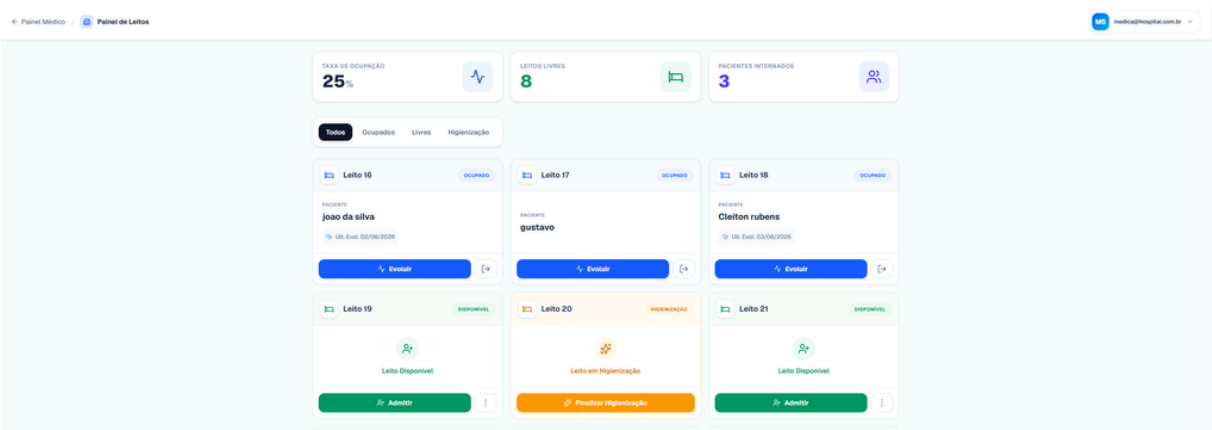
5.1.TELAS DO SISTEMA

Figura 1: Tela de Login



Legenda: Tela de acesso em formato split-screen. O painel esquerdo exibe a identidade visual do UTI Care com gradiente institucional; o painel direito contém o formulário de e-mail e senha. A identificação do papel do usuário é resolvida automaticamente pelo backend após autenticação — não há seletor manual de cargo.

Figura 2: Dashboard de Leitos (Painel do Médico/Enfermeiro)



Legenda: Grade de leitos da UTI com cards coloridos por status — verde para Vago, vermelho para Ocupado, amarelo para Higienização. Cada card exibe o número do leito, nome do paciente (quando internado), data de admissão e última evolução registrada. O cabeçalho apresenta contadores de KPIs: total de leitos, ocupados, vagos e em limpeza.

Figura 3: Prontuário do Paciente (/dashboard/[bedId])

Leito 18

paciente

Cleiton rubens

02/05/2028 10:30

OBSERVAÇÕES CLÍNICAS

card

Admissão Hospitalar

Entrada na UTI: 02/06/2028

Ações Clínicas

Nova Evolução

Histórico de Evoluções

02/06/2028 10:30:45

Dr(a): Maria Santos

EVOLUÇÃO MÉDICA - UTI

Data/Hora: 02/06/2028 09:47

Paciente: 70kg | Altura: Não informado

1. NEUROLÓGICO:

- Estado: Fatores obtido 17m/h 11.57 mcg/kg/h

- Reflexos: Incoercíveis e hiperreflexia

- BVS: BVS 40-60

- Neurol. Exame: Dócil

2. HEMODINÂMICO:

- Sem drogas vasoativas

- Sinais vitais: FMM: 65mmHg | FC: 47bpm | Ritmo: 47

- Perfusão: TPC: 36 | Lactato: 35 | SvO2: 25% | GapCO2: 36

- Oxi: SatO2 Normalmente

3. RESPIRATÓRIO:

- Via Aérea: Ar ambiente/VNI

- Sopro: Ar ambiente

- Oxigenação: SvO2: 12% | SvO2: 33%

- Rx Tórax: 34

- Oxi: 45

4. RENAL/METABÓLICO:

- Diurese: 24h: 54ml

- Metabólitos: HCT: 45

5. HEMATOINFECCIOSO:

- ATB em uso: 45 DDI

6. NUTRICIONAL:

- Dieta: Dieta oral

7. PROFILAXIAS:

- TEV: 45

- Quimiop: Não prescrito

- Outros: 45

Legenda: Tela de detalhe do leito com as informações do paciente internado: dados demográficos, data de admissão e observações clínicas. Exibe acordeão com histórico de evoluções clínicas anteriores e botões de ação: "Nova Evolução", "Editar Dados" e "Alta do Paciente".

Figura 4: Formulário de Evolução Clínica (Stepper)

Dados Gerais

Peso Predito (PBW)

Peso Real (kg)

-

90

Neurológico

Integridade neurológica e monitorização cerebral

Hemodinâmica

Perfusão, ritmo e drogas vasoativas

Sistema Respiratório

Problemas respiratórios e observações clínicas

TGI/Nutrição

Dieta, evacuações e aspecto abdominal

Renal/Metabólico

Diurese, função renal e controle glicêmico

Hematoinfeccioso

Antibiograma, cultura e curva térmica

Profilaxias

TEV, cicatriz de estresse e cuidado geral

Legenda: Formulário multi-etapas (stepper) com 7 passos, cada um correspondendo a um sistema fisiológico. A barra de progresso no topo indica o passo atual. O exemplo exibe o formulário Hemodinâmico com campos para drogas vasoativas (seletor com dose em mcg/kg/min), Pressão Arterial Média, Frequência Cardíaca, lactato e SvO2.

Figura 5: Modal de Resumo da Evolução

Resumo da Evolução

Revise o texto antes de salvar

EVOLUÇÃO MÉDICA - UTI

Data/Hora: 03/06/2026 15:57

Paciente: Peso não informado | Altura: 180cm

1. NEUROLÓGICO:

- Sem sedação contínua

- Pupilas: Fixas

- BIS: BIS 40-60

- Neuro Enteral: Dipirona, Quetiapina

2. HEMODINÂMICO:

- Sem drogas vasoativas

3. RESPIRATÓRIO:

- Via Aérea: Ar ambiente/VNI

- Suporte: Ar ambiente

4. RENAL/METABÓLICO:

5. HEMATOINFECIOSO:

- ATB em uso: Nenhum

6. NUTRICIONAL:

- Dieta: Zero/Suspensa

- TGI: VRG: Resíduo < 200 ml | Abdome: Abdome flácido, RHA+ | Evacuações: Fisiológicas

Copiar Texto

Confirmar e Salvar

Legenda: Modal de preview exibido antes de confirmar o salvamento da evolução. Apresenta o texto narrativo gerado automaticamente pelo sistema a partir de todos os campos preenchidos nos 7 sub-formulários, permitindo revisão antes da persistência no banco.

Figura 6: Painel Gerencial (/admin)

Dashboard de Administração

Bem-vindo ao painel central. Escolha uma das áreas abaixo para gerenciar.

Hospitais e Leitos

Gerencie as unidades hospitalares, visualize e crie novos leitos.

30 leitos no total

Equipe Hospitalar

Vincule médicos e enfermeiros aos hospitais específicos.

Gerenciar acessos

Funcionários

Cadastre e remova usuários do sistema (médicos, gestores, etc).

Controle de perfis

Visão Geral da Rede

TAXA DE OCUPAÇÃO

20%

PACIENTES ATIVOS

6

LEITOS LIVRES

23

EVOLUÇÕES HOJE

5

Legenda: Painel estratégico acessado por usuários com papel ADMIN ou MANAGER. Exibe KPIs de ocupação da UTI (taxa de ocupação, total de internações do dia, leitos em limpeza), lista de pacientes do dia, feed de atividades recentes e atalhos para ações gerenciais.

6.AMBIENTE E GUIA DE IMPLANTAÇÃO

6.1.REQUISITOS DO AMBIENTE

Componente	Versão Mínima	Observação
Sistema Operacional	Windows 10 / Ubuntu 22.04 LTS	Qualquer OS com suporte a Node.js
Node.js	v22.22.2	Definido no campo <code>engines</code> do <code>package.json</code>
npm	v10+	Incluído com Node.js
Git	v2.34+	Para clonar o repositório
Banco de Dados	PostgreSQL 15+	Pode ser local ou via Supabase (recomendado)
Conta Supabase	—	Necessária se usar o banco hospedado na nuvem

6.2.PROCESSO DE IMPLANTAÇÃO

1. Clonar o repositório:

```
git clone https://github.com/[usuario]/uti-clone-v0.git
cd uti-clone-v0
```

2. Instalar as dependências:

```
npm install
```

3. Configurar as variáveis de ambiente:

Crie um arquivo `.env` na raiz do projeto com o seguinte conteúdo:

```
# Conexão principal (via PgBouncer — porta 6543 — para uso em runtime)
DATABASE_URL="postgresql://postgres.[ref]:[senha]@aws-0-[regiao].pooler.supabase.com:6543/postgres?pgbouncer=true"

# Conexão direta (porta 5432 — para migrações Prisma)
DIRECT_URL="postgresql://postgres.[ref]:[senha]@aws-0-[regiao].pooler.supabase.com:5432/postgres"

# Chave secreta para assinar os tokens JWT (mínimo 32 caracteres)
SESSION_SECRET="sua-chave-secreta-com-minimo-32-caracteres-aqui"
```

4. Sincronizar o schema do banco de dados:

```
npx prisma db push
```

Este comando cria as tabelas no banco PostgreSQL conforme o `prisma/schema.prisma`. Use `npx prisma migrate deploy` em ambientes de produção com histórico de migrações.

5. Popular o banco com dados iniciais (opcional):

```
npx tsx prisma/seed.ts
```

6. Gerar o Prisma Client:

```
npx prisma generate
```

7. Iniciar o servidor de desenvolvimento:

```
npm run dev
```

Acesse `http://localhost:3000` no navegador.

8. Build para produção:

```
npm run build  
npm run start
```

Deploy na Vercel (recomendado):

1. Importe o repositório GitHub no dashboard da Vercel.
2. Configure as variáveis de ambiente (`DATABASE_URL`, `DIRECT_URL`, `SESSION_SECRET`) nas configurações do projeto.
3. O comando de build (`prisma generate && next build`) é executado automaticamente.
4. Cada push para a branch `main` dispara um novo deploy.

6.3.ACESSO À APLICAÇÃO IMPLANTADA

URL da Aplicação: `[INSERIR URL DA VERCEL — ex: https://uti-care.vercel.app]`

Credenciais de Acesso para Demonstração:

Papel	E-mail	Senha	Acesso
Médico (DOCTOR)	medico@uticare.com	[INSERIR]	/dashboard — painel clínico
Enfermeiro (NURSE)	enfermeiro@uticare.com	[INSERIR]	/dashboard — gestão de leitos
Gestor (MANAGER)	gestor@uticare.com	[INSERIR]	/admin — painel gerencial

Nota: Para criar usuários de teste via seed, execute `npx tsx prisma/seed.ts` após configurar o banco.

7. CONCLUSÃO

O desenvolvimento do **UTI Care** consolidou a aplicação prática dos fundamentos de engenharia de software em um domínio de alto impacto clínico. O sistema resultante é uma aplicação web full-stack funcional que digitaliza dois fluxos críticos de uma UTI: a gestão do ciclo de vida dos leitos (admissão → alta → higienização) e o registro estruturado de evoluções clínicas multi-sistemas.

A integração bem-sucedida de tecnologias modernas — Next.js App Router, Prisma ORM com PostgreSQL, sessões JWT e RBAC via middleware — demonstrou que é possível construir um sistema com camadas claras de segurança e separação de responsabilidades sem a complexidade operacional de manter dois projetos separados (frontend + backend).

7.1. TRABALHOS FUTUROS

- **Módulo de Gestão de Equipe** (`/admin/staff`): Permitir que o gestor cadastre, visualize e desative colaboradores diretamente pela interface, sem depender de scripts de seed manuais.
- **Auditoria de Evoluções**: Tela para a gestão verificar se todos os pacientes internados receberam evolução clínica nas últimas 24h, garantindo conformidade regulatória.
- **Linha do Tempo Completa no Prontuário**: Exibir todas as evoluções anteriores do paciente em uma timeline navegável diretamente no prontuário (`/dashboard/[bedId]`).
- **Painel de Indicadores com Gráficos**: Integrar uma biblioteca de visualização (ex: Recharts) para exibir gráficos de taxa de ocupação mensal, média de permanência em leito e percentual de altas vs óbitos.
- **Rascunhos de Evolução (DraftEvolution)**: Implementar salvamento automático do formulário de evolução como rascunho no banco, prevenindo perda de dados em caso de navegação acidental.
- **Prescrições Clínicas Simplificadas**: Módulo para registro de condutas medicamentosas urgentes (drogas vasoativas, sedativos, antibióticos) vinculadas ao paciente.
- **Aplicativo Mobile (PWA)**: Converter a aplicação em um Progressive Web App para uso em tablets à beira do leito, eliminando a dependência de estações de trabalho fixas.
- **Integração com sistemas legados**: Exportação de dados no padrão HL7 FHIR para integração com prontuários eletrônicos hospitalares já existentes.

7.2. LIÇÕES APRENDIDAS

O desenvolvimento do UTI Care foi uma jornada de aprendizado técnico e colaborativo de alta intensidade. O maior desafio técnico foi a modelagem das transações atômicas para o ciclo de vida dos leitos: garantir que a criação do paciente e a atualização do leito ocorressem como uma operação indivisível — evitando estados inconsistentes — exigiu compreensão profunda de transações Prisma com `$transaction`.

A decisão de usar Next.js Server Actions como camada de backend simplificou enormemente o desenvolvimento inicial, mas revelou um desafio de testabilidade: actions que chamam `redirect()` internamente são difíceis de testar com Jest convencional. A lição foi que a testabilidade deve ser considerada antes da implementação, separando a lógica de negócio da camada de navegação.

Em termos de gestão, aprendemos que a documentação técnica produzida durante o desenvolvimento (e não ao final) economiza um tempo substancial: o relatório técnico incremental mantido ao longo do projeto tornou este documento final muito mais ágil de produzir, pois as decisões arquiteturais já estavam registradas com contexto e justificativa.

A principal lição foi que segurança não pode ser adicionada depois: implementar RBAC, JWT e bcrypt desde o início como requisitos não-negociáveis definiu a qualidade do sistema muito mais do que qualquer funcionalidade adicional poderia ter feito.
